Memory Efficient Doubly Linked List – Delete At Begining –Space Complexity Analysis and Algorithm

Program:

```
void deleteFromBeginning()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *temp = head;
    // Decode the second node (which will become head)
    // Next = NULL ^ head->npx
    Node *next = XOR(nullptr, head->npx);

    if (next != nullptr)
    {
        // Update the new head's npx.
        // Currently next->npx is: XOR(head, nextNext)
        // We need it to be: XOR(NULL, nextNext)
        Node *nextNext = XOR(temp, next->npx);
        next->npx = XOR(nullptr, nextNext);
    }

    head = next;
    cout << "Deleted " << temp->data << " from the
beginning." << endl;
    free(temp);
}
```

Space Complexity Analysis: *XOR Linked List* (*Delete From Beginning*)

1. *Input Space Complexity*

- ***Parameter − only Definition***:
  - ***Function Parameters***: *None* (*takes no arguments*).
  - ***Analysis***: *Since no data is passed into the function, the parameter space is zero.*
  - ***Complexity***: $O(1)$

- ***Full Data Definition***:
  - ***Function Data***: *The existing XOR linked list structure* (*specifically the head and its immediate successor*).
  - ***Analysis***: *The function operates on a list containing n nodes. The "input" context is the list itself.*
  - ***Complexity***: $O(n)$

---

2. *Auxiliary Space Complexity*

Definition: *This measures the temporary memory used by the function to perform the deletion logic.*

- ***Local Variables***: *The function uses three local pointers*: *temp*, *next*, *and nextNext*.
  - ***Analysis***: *These are stack − allocated and take a fixed amount of memory* (*constant*)*regardless of how many nodes are in the list.*

- ***XOR Operations***: *The bitwise XOR calculations are performed within CPU registers.*

- ***Memory Management***: *The free*(*temp*)*operation releases memory back to the system. While this changes the total memory used by the program, it does not require extra auxiliary space to perform the deallocation.*

- ***No Recursion***: *The logic is straightforward and iterative, so there is no growth in the call stack.*

Conclusion: *The extra space required to perform the deletion is constant.*

- ***Auxiliary Space Complexity***: $O(1)$

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\boldsymbol{Space} = \underbrace{\boldsymbol{O(1)}}_{\boldsymbol{Auxiliary}} + \underbrace{\boldsymbol{O(1)}}_{\boldsymbol{Input}} = \boldsymbol{O(1)}.$$

- **Holistic Interpretation (Full-data input):**

$$\boldsymbol{Space} = \underbrace{\boldsymbol{O(1)}}_{\boldsymbol{Auxiliary}} + \underbrace{\boldsymbol{O(n)}}_{\boldsymbol{Input}} = \boldsymbol{O(n)}.$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

Algorithm: *Delete From Beginning* (*XOR Doubly Linked List*)

XORDELBEG(***START***):

1. [***CHECK IF LIST IS EMPTY***]
   $If\ START\ =\ NULL, then$:

   $Write$: "$List\ is\ empty. Nothing\ to\ delete.$"

   $Return\ and\ \boldsymbol{EXIT}.$

   [$End\ of\ If\ structure$]

2. [***INITIALIZE TEMPORARY POINTER***]
   $Set\ TEMPPTR := START$ ($Store\ the\ current\ head\ to\ free\ its\ memory\ later$)

3. [***DECODE THE SECOND NODE***]

   $Set\ NEXTTEMPPTR := NULL \oplus NEXTXORPTR[START]$ ($Calculate\ the\ address\ of\ the\ node\ that\ will\ become\ the\ new\ head$)

4. [***UPDATE NEW HEAD'S LINKAGE***]

   $If\ NEXTTEMPPTR \neq NULL, then$: ($If\ the\ list\ has\ more\ than\ one\ node,\ we\ must\ update\ the\ successor's\ NPX$)

   $a.\ Set\ NEXTNEXTTEMPPTR$:
   $= TEMPPTR$
   $\oplus\ NEXTXORPTR[NEXTTEMPPTR]$ ($Decode\ the\ node\ following\ the\ second\ node\ using\ the\ current\ head\ as\ the\ 'previous'\ reference$)

   $b.\ Set\ NEXTXORPTR[NEXTTEMPPTR]$:
   $= NULL \oplus NEXTNEXTTEMPPTR$ ($Update\ the\ new\ head's\ NPX\ to\ indicate\ its\ previous\ neighbor\ is\ now\ NULL$)

   [$End\ of\ If\ structure$]

5. [***UPDATE GLOBAL START***]

   $Set\ START := NEXTTEMPPTR \left(\begin{array}{c} The\ second\ node\ officially \\ becomes\ the \\ first\ node \\ of\ the\ list \end{array} \right)$

6. [***FREE MEMORY AND PRINT***]

   $Write$: Deleted , $INFO[TEMPPTR]$, from the beginning.

   $\boldsymbol{FREE}(TEMPPTR) \left(\begin{array}{c} Return\ the\ memory\ of\ the \\ deleted\ node\ to\ the\ system \end{array} \right)$

7. ***EXIT***
